menler

CLAUDE-FIRST FELLOWSHIP · WEEK 5

SOLUTION

Capstone scope & MVP skeleton

A reference scope and skeleton for StudyLoop — an app that turns a student's notes into an adaptive quiz. One brief, one CLAUDE.md, one live route. Yours can differ.

Week 5 · Claude Code · CLAUDE.md · Supabase · Replit · GitHub

Reference build — yours can differ.

W5

Capstone scope & MVP skeleton

How it's executed

THE EXECUTION — HOW, WHERE & WHEN

The worked product is **StudyLoop**: a student pastes their notes, and the app uses Claude to generate a short quiz, then adapts the next round to the questions they got wrong. The **scope** fits on one page. The **build** happens in Claude Code, which reads a CLAUDE.md for context on every prompt. The **data** lives in Supabase. The **first route** — paste notes, get one question back — is deployed on Replit so it is reachable from a phone.

The point of the week is not a finished app. It is proving the **foundation holds**: a scope tight enough to code against, a CLAUDE.md Claude actually follows, and one real route live on the internet — so next week is building, not deciding.

WHAT TOOLS — AND THEIR ROLE

Claude Code	The builder. Reads CLAUDE.md, scaffolds the repo, writes the first route, and debugs it with you.
CLAUDE.md	The context file. Goal, stack, conventions and "never do" rules — read by Claude on every prompt.
Supabase	The database. Stores notes, generated questions, and which answers were wrong.
Replit	The deploy. Hosts the first live route at a URL you can open anywhere.
GitHub	The checkpoint. Every working state committed, so a bad change is one revert away.
MCP (Claude API)	The intelligence at runtime. The app calls Claude to turn notes into a question.

THE WORKED SOLUTION

Do it — *step by step (1 of 2)*

1 Write the one-page brief

DO THIS

- Four short blocks: **Problem** (students re-read notes instead of testing themselves), **User** (one: a first-year revising for a test), **Solution** (paste notes → adaptive quiz), **Success** (a user finishes a 5-question round and sees what to revise).
- Keep it to a page. If it needs two, the scope is too big.

Done when: a stranger could read it and tell you what the app does and who it is for.

2 Choose and justify the stack

DO THIS

- Pick a tool per layer and write one honest line of why.
- Favour tools that let one person ship — not what a ten-person team would choose.

```
STACK — WITH REASONS
Frontend: Lovable  — fastest way to a usable UI solo
Backend:  Replit   — build + host in one place
Data:     Supabase — free tier, auth + DB included
Brain:    Claude   — notes to questions, one API call
```

Done when: every line answers "why this?" and not just "what".

3 Design the Claude architecture

DO THIS

- Write the app's **system prompt** — one job: turn notes into one clear question with a correct answer.
- List the **MCP tools** it needs (here: Supabase read/write) and any **Routine** (a nightly "revise this" nudge). Decide what Claude does at runtime vs what you pre-build.

Done when: you can name exactly where Claude sits in the app and what it is allowed to do.

THE WORKED SOLUTION

Do it — *step by step (2 of 2)*

4 Write CLAUDE.md and test it

DO THIS

- Put the goal, the stack, the conventions and the "never" list in a CLAUDE.md at the repo root.
- Open Claude Code, ask it to **summarise the project back to you**, and fix the file wherever it guessed wrong.

CLAUDE.MD — SKELETON

```
# StudyLoop
Goal: paste notes -> adaptive 5-question quiz.
Stack: Lovable + Replit + Supabase + Claude API.
Conventions: TypeScript, small components, one route per file.
Never: store notes without consent; block the UI on an API call.
```

Done when: Claude's summary of the project matches what you meant.

5 Scaffold the repo

DO THIS

- Ask Claude Code to generate the folders, the first component, and an empty "paste notes" route from the CLAUDE.md.
- Commit the scaffold to GitHub before you change anything — that is your first checkpoint.

Done when: the repo runs locally and shows the paste-notes screen.

Tip — let Claude generate; you review. Read every file it writes before you commit it.

6 Deploy the first route live

DO THIS

- Wire the route to one Claude call: notes in → one question out. Store the note in Supabase.
- Deploy on Replit and open the URL on your phone. Paste a paragraph; get a real question back.

Done when: a public URL takes your notes and returns one question — live, on a phone.

YOUR OUTPUT — CLAUDE.MD

```
# StudyLoop
Goal: turn a student's pasted notes into an adaptive
5-question quiz that re-tests what they got wrong.

Stack:
  Frontend  Lovable      (solo-fast UI)
  Backend   Replit       (build + host)
  Data      Supabase     (notes, questions, wrong answers)
  Brain     Claude API   (notes -> one question + answer)

Conventions: TypeScript; small components; one route/file.
Never: store notes without consent; block the UI on a call.
```

TOOL OVERVIEW

What you'll find *where*

Every surface this scope-and-skeleton uses, and the one job it does.

CLAUDE.md	Repo root. The context Claude Code reads on every prompt. Edit it when the plan changes.
Claude Code	Your terminal or editor. Where the scaffold and the first route get written and debugged.
Connectors (MCP)	Customize → Connectors. GitHub and Supabase linked so Claude can act on the repo and the data.
Supabase dashboard	Your Supabase project. Tables for notes and questions; the free tier is enough.
Replit	Your Replit workspace. The deploy that gives the first route a public URL.
GitHub repo	Your repository. Every commit is a checkpoint you can return to.

MAKE IT REUSABLE

Set it up & *reuse it*

Scoping and scaffolding is a skill you repeat for every build. These three surfaces make the next one faster.

SET UP CLAUDE CODE

Where: Install Claude Code, open your repo, and add a CLAUDE.md at the root.

1. Put goal, stack, conventions and "never" rules in CLAUDE.md.
2. Ask Claude to summarise the project back; fix what it gets wrong.
3. Keep the file updated as the plan changes — stale context causes drift.

For this project: StudyLoop's CLAUDE.md is the single source of truth Claude builds against — scope lives in the repo, not your head.

A good CLAUDE.md is the difference between Claude helping and Claude guessing.

CONNECT AN MCP

Where: Customize → Connectors → + → pick the app → sign in → Allow. Switch it on in the build chat.

1. Connect **GitHub** so Claude can read and commit to the repo.
2. Connect **Supabase** so it can create tables and queries.
3. Switch both on inside the Claude Code session.

For this project: Claude scaffolds the repo and sets up the notes table without you leaving the chat.

CREATE A SKILL

Where: Customize → Skills → New skill. A saved instruction Claude reuses on command.

1. Save a "scaffold a new route from the CLAUDE.md" instruction as a skill.
2. Name it so it is obvious when to use it.
3. Reuse it for every new route this week and next.

For this project: adding the "take a quiz" route next week is one skill call, not a from-scratch prompt.

PROMPT LIBRARY · STAR

Write it in *STAR*

Four levels of the scope-and-scaffold prompt — Situation, Task, Action, Result. Start basic, grow into pro.

BASIC

- S **Situation.** I have an app idea but no plan and a blank repo.
- T **Task.** Help me write a one-page scope for it.
- A **Action.** Ask me the problem, the one user, the solution and what "done" means, then draft the brief.
- R **Result.** A one-page scope I can actually build against.

INTERMEDIATE

- S **Situation.** I have a scope and need the right stack and a repo.
- T **Task.** Recommend a solo-friendly stack and scaffold the repo.
- A **Action.** Suggest frontend, backend, data and deploy with one reason each; then generate folders and a first route.
- R **Result.** A justified stack and a repo that runs locally.

ADVANCED

- S **Situation.** I want Claude to stay on-spec while it builds, not drift.
- T **Task.** Set up a CLAUDE.md and build from it.
- A **Action.** Write goal, stack, conventions and "never" rules; summarise the project back; then scaffold strictly from the file.
- R **Result.** A CLAUDE.md Claude follows, and a scaffold that matches the plan.

PRO

- S **Situation.** I need the first route live, with Claude in the loop, by end of week.
- T **Task.** Wire one route to a Claude call and deploy it.
- A **Action.** Build notes-in / question-out, store to Supabase, deploy on Replit, and commit each working state to GitHub.
- R **Result.** A public URL that runs real AI — the launch pad for next week's sprint.

WHY IT MATTERS

Where this pays off *in real life*

Scope-then-scaffold is how professionals start every build. The habit outlasts this one app.

- **Ship an MVP** — turn a weekend idea into a running product instead of a folder of notes.
- **A college tool** — a study aid, a timetable helper, a club directory that your campus actually uses.
- **An internal tool** — scope a small app for a team at work and have Claude build the first version.
- **A startup prototype** — a scope doc and a live route are enough to show an investor the idea is real.

WHAT GREAT LOOKS LIKE

- The scope fits on **one page** and a stranger understands it.
- Every stack choice has an **honest reason**, not just a name.
- The CLAUDE.md is tested — Claude's summary **matches your intent**.
- One route is **live on a URL**, with Claude actually running in it.